

Wane Enterprises

HubSpot - Stripe Bi-Directional Integration Architecture

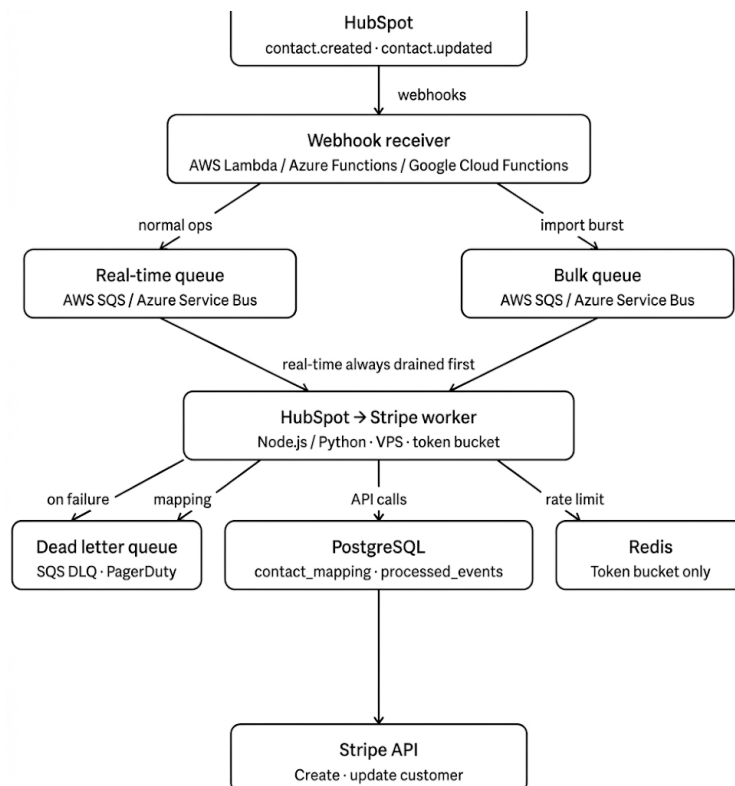
Summary:

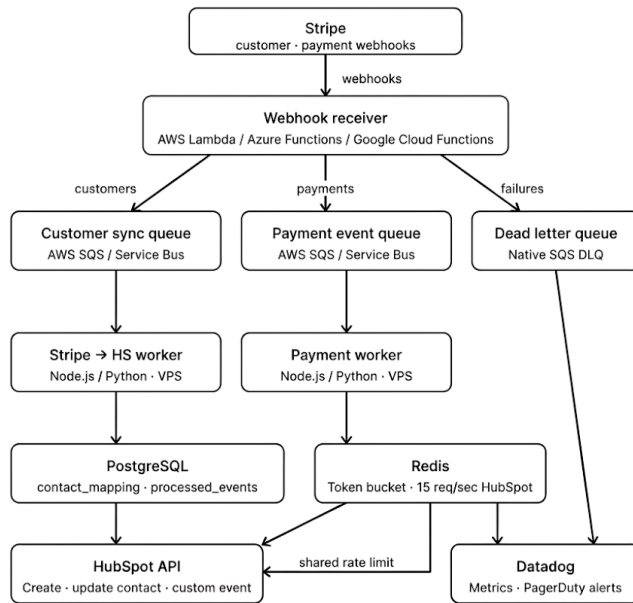
This solution provides near real-time bi-directional synchronization between HubSpot and Stripe, maintaining data consistency, fault tolerance, scalability during import spikes, and protection against duplicate processing.

The architecture is event-driven, built on serverless webhook receivers, message queues, worker services, and a centralized mapping database.

One constraint exists: during occasional large-scale HubSpot imports of up to 10,000 contacts in a single minute, Stripe's enforced API limit of 100 calls per second means the full batch cannot be synced within 30 seconds. All contacts will be synced - just not all within the SLA window. This is addressed in the scalability section, and I feel it should be formally agreed with the client as a documented exception prior to go-live.

High-Level Architecture and flow:





Core Components

1. Webhook Receiver (AWS Lambda / Azure Functions / Google Cloud Functions)

Receives events from HubSpot and Stripe. Validates the webhook signature, checks the processed_events table in PostgreSQL for duplicate event IDs and discards if already seen, discards any event tagged as originating from our own integration, then enqueues the payload and returns HTTP 200 immediately. Receivers never call downstream APIs directly - this keeps them fast and resilient to bursts.

2. Priority Event Queue (AWS SQS / Azure Service Bus)

Two queues per direction: a real-time queue for normal operations and a bulk queue for import events. If more than 500 HubSpot contact-created events arrive within 60 seconds, they are classified as an import and routed to the bulk queue. Workers always drain the real-time queue first, ensuring the 30-second SLA holds for normal activity throughout any import.

A separate queue handles Stripe payment events to prevent high-volume payment traffic from starving customer sync events. Failed messages retry 3 times before routing to the Dead Letter Queue.

3. Worker Services (Node.js or Python on VPS)

Three workers, one per flow:

- **HubSpot → Stripe Worker:** On contact created, checks PostgreSQL for an existing mapping before calling the Stripe Create Customer API. On contact updated, patches only the changed fields. Respects Stripe's 100 req/sec limit via a shared Redis token bucket.
- **Stripe → HubSpot Worker:** On customer created, searches HubSpot by email first to prevent duplicate contacts. On customer updated, patches only changed fields. Respects HubSpot's 15 req/sec limit via a shared Redis token bucket.
- **Payment Event Worker:** On payment succeeded, looks up the HubSpot contact ID from PostgreSQL using the Stripe customer ID, then calls the HubSpot Custom Event API. If the mapping is not yet available (race condition: customer was just created), the message is requeued with a 10-second delay and retried up to 3 times before falling back to a direct HubSpot email search.

4. Mapping Database (PostgreSQL)

Stores the permanent relationship between HubSpot contact IDs and Stripe customer IDs, plus a `processed_events` table as a permanent audit log of every webhook received.

PostgreSQL is chosen over Redis because this is permanent business-critical data that must never be lost. Redis is an in-memory store and carries operational risk not appropriate for data of this importance. Redis is used only as the shared rate limiter token bucket.

Email is not used as the primary key because email addresses can change, which would orphan records. Persistent system-assigned IDs are used instead.

`contact_mapping`

```
-----
hubspot_contact_id  VARCHAR  PRIMARY KEY
stripe_customer_id  VARCHAR  UNIQUE NOT NULL
email               VARCHAR
created_at          TIMESTAMP
last_synced_at      TIMESTAMP
last_source         VARCHAR
version             INTEGER
```

`processed_events`

```
-----
event_id           VARCHAR  PRIMARY KEY
source            VARCHAR
processed_at       TIMESTAMP
```

Event Types

Trigger	Action
HubSpot contact created	Create Stripe customer
HubSpot contact updated	Update Stripe customer
Stripe customer created	Create HubSpot contact
Stripe customer updated	Update HubSpot contact
Stripe payment succeeded	Log custom event on HubSpot contact

Loop Prevention

Bi-directional webhooks create a risk of infinite update cycles. Stopped with two layers:

Layer 1- Origin tagging: Every write our system makes is stamped with a source marker — `metadata.source = "hubspot-sync"` on Stripe writes, `integration_source = "stripe-sync"` on HubSpot writes. Incoming webhooks carrying our own tag are discarded immediately.

Layer 2- Idempotency: Every webhook carries a unique event ID recorded permanently in the PostgreSQL `processed_events` table. Duplicate deliveries are detected and dropped before any processing occurs.

Rate Limits

System	Limit
Stripe	100 req/sec
HubSpot	15 req/sec

HubSpot's 15 req/sec is the binding constraint. All workers writing to HubSpot share a single Redis-backed token bucket, ensuring the combined rate never exceeds the limit across all worker instances. HTTP 429 responses trigger a back-off using the Retry-After header without consuming a retry attempt.

The HubSpot batch API is not used - a single invalid record fails the entire batch of 100 with no per-record error reporting. Individual API calls are safer and the volume does not justify the risk.

Scalability

Event Type	Average	Peak
HubSpot contact creates/updates	1,000/day	10,000 in one minute
Stripe customer creates	1,000/day	100/minute
Stripe customer updates	10/day	negligible
Stripe payments	10,000/day	150/minute

The import burst of 10,000 contacts in one minute (~167/sec) exceeds Stripe's 100 req/sec limit. All events are safely enqueued and none are lost. The bulk queue drains at 100/sec, completing in approximately 100 seconds. Real-time events are unaffected throughout. No infrastructure change can resolve this - the limit is enforced by Stripe at the account level.

Error Handling

Attempt	Delay
1st retry	5 seconds
2nd retry	30 seconds
3rd retry	2 minutes
Dead Letter Queue Manual review + immediate alert	

Security

- Webhook signatures validated on every inbound request (HMAC-SHA256 for Stripe, HubSpot signature header for HubSpot). Invalid requests rejected with HTTP 401.
- All traffic over HTTPS / TLS 1.2+
- All data encrypted at rest via cloud provider managed key service
- API keys and secrets stored in AWS Secrets Manager / Azure Key Vault - never in code or environment variables
- Workers, PostgreSQL, and Redis deployed inside a private VPC with no public IP addresses. Outbound traffic routed via NAT gateway.
- PII masked in all log output

Monitoring & Alerting

Condition	Severity
Any message enters DLQ	Critical

Condition	Severity
Queue message age > 20 seconds	Warning
Queue message age > 28 seconds	Critical
PostgreSQL unavailable	Critical
Redis unavailable	Critical
Rate limit utilization > 80%	Warning

Technology Stack

Layer	Technology
Webhook Receivers	AWS Lambda / Azure Functions /Google Cloud Functions
Queue	AWS SQS / Azure Service Bus / Google Pub/Sub
Mapping Database	PostgreSQL
Rate Limiter	Redis
Worker Services	Node.js or Python on VPS
Secrets	AWS Secrets Manager / Azure Key Vault
Monitoring	Datadog or CloudWatch
Alerting	PagerDuty
Dead Letter Queue	Native DLQ of chosen queue provider